

Knowing Where, Not Going There: Vision-Language-Action Policies Encode Displacements They Do Not Act On

Anonymous CVPR submission

Paper ID *****

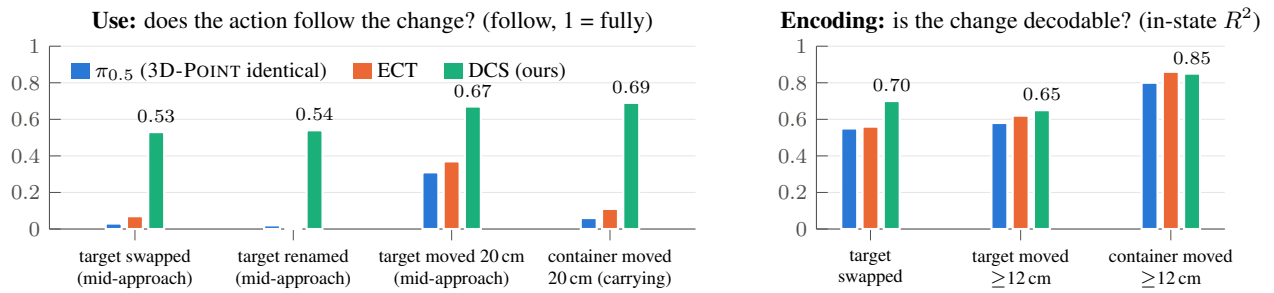


Figure 1. $\pi_{0.5}$ **knows where, but does not go there**. Probes at decision states of LIBERO-Object rollouts (Sec. 3). *Right*: the displacement of the target or the container is decodable from the action tokens of every model. *Left*: at the moments that decide the task, the action of $\pi_{0.5}$ barely follows it; neither equivariant counterfactual training (ECT) nor an injected oracle 3D goal point (3D-POINT, identical to $\pi_{0.5}$) changes that. Decision-time counterfactual supervision (DCS) does, without changing the encoding.

Abstract

Vision-language-action (VLA) policies that are near-perfect on LIBERO collapse once the objects in the scene trade places: $\pi_{0.5}$ completes 99% of LIBERO-Object episodes but only 19% after two objects are swapped, driving to where the target used to be. We ask whether this is a failure of perception or of use. Intervening on the simulator at arbitrary states of the policy’s own rollouts, we measure, phase by phase, how far the predicted action follows a displacement of the target or the container (use) and how well that displacement can be decoded from the policy’s hidden states (encoding). $\pi_{0.5}$ encodes the displacement (in-state probe R^2 0.55–0.86), yet at the moments the task is decided its action follows it by only 3–22%. Two recent remedies leave use unchanged in a matched setting: equivariant counterfactual training on transformed demonstrations, and injecting a privileged 3D goal point into the action expert, which is learned as a constant bias. Supervising the policy in counterfactual worlds built at its own decision states, with privileged labels gated for validity, raises use to 0.44–0.69 while encoding stays where it was. Restricting which parameters may change localizes the effect: adapting only

the VLM backbone, mostly its layers 9–17, recovers about 90% of the gain, whereas the action expert, its readout, or the VLM’s keys and values alone do not. Under the same teacher and training budget, this improves LIBERO-PRO object swaps by 30.8 points on LIBERO-Object and 14.3 points on LIBERO-Spatial over equivariant counterfactual training, at unchanged nominal success. We also report what it costs: a trade-off with appearance changes, and a sensitivity to the furniture placement that LIBERO re-draws at every reset, an evaluation detail we show moves per-task success by up to 25 points.

1. Introduction

Generalist vision-language-action (VLA) policies [5, 6, 13, 18] now solve the standard LIBERO suites [16] almost perfectly. The same policies fail in a strikingly simple way when the scene is rearranged. LIBERO-PRO [26] keeps every task and only moves the objects; under its *swap* perturbation, where the target trades places with another object, $\pi_{0.5}$ drops from 99% to 19% on LIBERO-Object and from 98% to 40% on LIBERO-Spatial. Watching the rollouts shows why: the arm drives to where the target used

to be. The policy behaves as if the instruction retrieved a memorized trajectory [7].

A natural reading is that the policy does not *perceive* where things are, and a large body of work adds perception: depth, point clouds, 3D features, or an explicit 3D goal point [20, 22, 25]. We test this reading directly and find it wrong for $\pi_{0.5}$. Because our policies run in a simulator, we can stop any rollout at any state, change exactly one thing (move the target, swap it with another object, move the container while the object is carried, rename the target in the instruction), re-render, and ask the policy again. Comparing the two predictions measures how much the action *uses* the change; probing the hidden states before and after the change measures how well the change is *encoded*. The two disagree (Fig. 1). The displacement is linearly decodable from the action tokens of $\pi_{0.5}$ (R^2 0.55–0.86), but at the moments that decide the task, when the hand is still on its way to the target or carries the object to the container, the predicted action follows the displacement by only 3–22%. $\pi_{0.5}$ *knows where* the target is and does not *go there*.

This distinction predicts which remedies can work. If the information is already present, adding more of it should not help, and it does not: re-implementing the injection of a privileged 3D goal point into the action expert [22] in our setting leaves swap success at the base level, and the injected channel is learned as a constant offset that ignores the point. Equivariant counterfactual training (ECT) [7], which pairs demonstrations with geometrically transformed replays, diagnoses the same memorization but, in a matched setting, leaves the measured use where it was. What does change use is supervision *at the decision states themselves*: building, at a state of the policy’s own rollout, a world in which the memorized action is wrong (the target or the container has moved) and supervising the correct action there. We call this decision-time counterfactual supervision (DCS). It raises use from 0.03–0.09 to 0.44–0.69 while the encoding does not change: it teaches the policy to act on what it already represents rather than to see better.

The same instrument tells us *where* the change happens. Restricting the trainable parameters to different parts of $\pi_{0.5}$, with the same counterfactual data, shows that the action expert, its readout layer, or the keys and values the action expert reads from the VLM are not where the fix lives; adapting the VLM backbone alone, with the action expert frozen, recovers about 90% of the swap gain, and most of that comes from its layers 9–17.

Contributions.

- A decision-time intervention protocol that separates *use* from *encoding* in a VLA, per task phase, and the finding that $\pi_{0.5}$ encodes object and container displacements it does not act on (Sec. 3).
- Evidence that two recent remedies do not change use in

a matched setting, including privileged 3D goal injection, whose channel collapses to a constant: geometric information is not what is missing (Sec. 5.2).

- A localization of the change that does fix use: the VLM’s mid-to-late layers, not the action pathway (Sec. 5.3).
- Decision-time counterfactual supervision with validity-gated privileged labels, which improves LIBERO-PRO swap by 30.8 points (LIBERO-Object) and 14.3 points (LIBERO-Spatial) over ECT under the same teacher and budget (Sec. 5.1), together with an account of its costs and of an evaluation detail of LIBERO, the per-reset replacement of furniture, that changes per-task results by up to 25 points (Sec. 5.4).

2. Related Work

VLA policies and their generalization. VLAs fine-tune vision-language backbones to output robot actions, as discrete tokens [6, 13] or through a flow-matching action expert attached to the backbone [5, 18]; continuous-action fine-tuning brought them close to saturation on LIBERO [14, 16]. Robustness benchmarks built on LIBERO show how narrow this success is: LIBERO-Plus [10] perturbs camera, lighting, robot initial state and layout, and LIBERO-PRO [26] swaps and moves objects, rephrases instructions and changes tasks, with near-zero success for most models under position changes. We study the position failures of $\pi_{0.5}$, the strongest open model on these suites.

Memorization and shortcuts in imitation. Behavior cloning is known to latch onto spurious correlates of the expert’s action, such as past actions or proprioception [9, 23], an instance of shortcut learning [11]. For VLAs, recent analyses report that instructions retrieve memorized trajectories [7], that new target positions remain decodable from internal features while the arm goes to the old one [2], and that spatial motor programs are tied to the scene layout [3]. Our measurements agree with this diagnosis; what we add is a per-phase, per-intervention separation of use and encoding that compares models before and after a remedy, and a localization of the parameters whose change restores use.

Adding geometry. Many works feed depth, point clouds or 3D features to VLAs [20, 25]; where position perturbations were measured, they did not resolve them. Closest to our question, Tsai *et al.* [22] inject a grounded 3D goal point into the adaptive normalization of the action expert and report large gains when training on 69 tasks with varied layouts. We re-implement the injection with an oracle point in our per-suite setting and find that the channel is ignored; with counterfactual supervision added, the policy still solves the task through the VLM rather than through

the point.

Counterfactual data and privileged teachers. Counterfactual data augmentation [19] and demonstration generation by replaying transformed trajectories [17, 24] create new supervision from existing episodes. ECT [7] is the closest method: it mirrors or shifts the whole scene at the start of a demonstration and replays the transformed end-effector path, training on both. Our counterfactuals are instead built at *arbitrary states of on-policy rollouts*, one change at a time (the target before the grasp, the container while carrying), and labeled by a privileged scripted teacher, in the spirit of learning from privileged experts [8] and on-policy distillation [1, 21]. We use them as an instrument to ask where VLAs fail, and treat the resulting training procedure as the minimal intervention the analysis calls for rather than as a data-generation contribution.

3. Does $\pi_{0.5}$ See the Displacement, and Does It Use It?

Model. $\pi_{0.5}$ [18] couples a PaliGemma VLM [4] (18 layers, width 2048) with an action expert (18 layers, width 1024) that produces a 50-step action chunk by flow matching [15]. At every layer the action tokens attend to the VLM’s prefix: 512 image tokens from an agent-view and a wrist camera, the instruction, and the proprioceptive state, which the LIBERO checkpoint writes into the prompt as about 120 digit tokens. The time step of the flow conditions the expert through adaptive RMS normalization. We use the public LIBERO checkpoint in its LeRobot port; on LIBERO-PRO it matches the numbers reported for the official model (swap: Object 18.7 vs. 18.2, Long 8.5 vs. 9.4 [7]).

Decision-time interventions. We take states s_t from successful rollouts of the policy and build a counterfactual state s'_t that differs in one respect, by editing the simulator’s kinematic state and re-rendering both cameras: (i) before the grasp, the target is moved by 3, 6, 12 or 20 cm to a free spot, or (ii) it swaps places with another object; (iii) while the object is carried, the container is moved by the same amounts; (iv) before the grasp, the instruction names another object of the scene; (v) only the proprioceptive input is offset by 5 cm. Nothing else changes, the robot included. States are binned by phase: *early* (hand farther than 15 cm from the target), *mid* (5–15 cm), *near* (closer than 5 cm) and *carrying*.

Use. With the flow noise held fixed, we roll the first 25 steps of each predicted chunk through a kinematic model of the controller to obtain the hand’s intended displacement

Phase	3 cm	20 cm	swap	rename	proprio
early (>15 cm)	0.18	0.22	0.22	0.25	0.00
mid (5–15 cm)	0.61	0.31	0.03	0.02	0.00
near (<5 cm)	0.51	0.29	0.01	−0.02	0.00
carrying	0.09	0.06	−	−	0.00

Table 1. **Use of a displacement by $\pi_{0.5}$** , LIBERO-Object (median over states). Columns: target moved by 3 or 20 cm (container while carrying), target swapped with another object, target renamed in the instruction, proprioception offset by 5 cm. At the decision moments (bold) the action barely follows the change.

$m(s)$, and define

$$\text{use} = \frac{\langle m(s'_t) - m(s_t), \mathbf{d} \rangle}{\|\mathbf{d}\|^2}, \quad (1)$$

where $\mathbf{d}$ is the displacement of what moved (for (iv), from the target to the newly named object). A value of 1 means the motion follows the change fully, 0 that it ignores it.

Encoding. For every layer we pool the hidden states of four streams (image tokens, instruction tokens, the last prefix token and the action tokens) and regress the *within-state* difference $h(s'_t) - h(s_t)$ on $\mathbf{d}$ with ridge regression, reporting cross-validated R^2 with folds grouped by episode. Differencing the same state removes everything that varies across states (where the hand is, the time in the episode), so the probe only asks whether the stream registers where the object was moved to.

Findings. Table 1 reads as follows. (1) *At the decision moments the policy is nearly blind.* Early in the approach the action follows a moved target by about 20%; mid-way it ignores a swap (0.03) or a renamed target (0.02) entirely, and while carrying it follows a moved container by 6–9%: placing is a memorized motion. (2) *Once close, it is a local tracker.* Small displacements near the hand are followed (0.51–0.61) more than large ones, but the choice of object is locked. (3) *Proprioception is not the shortcut.* Offsetting the proprioceptive input has no effect in any phase, and replacing it by a constant at test time (Object swap 14.0, nominal 100) or during training (Spatial swap 40.5 vs. 40.0) does not change the failure. The same pattern holds on LIBERO-Spatial (early 0.21–0.30, swap 0.08–0.14, carrying 0.17–0.30) and LIBERO-Goal.

At the same time, the displacement is *encoded*. It is decodable from the image tokens at every layer (R^2 0.7–0.8 for a target moved by 12 cm or more, about 0.9 for a moved container) and is carried into the action tokens over the expert’s layers, from about 0.05 in its first layers to 0.58 (target) and 0.74–0.82 (container) in its last ones. The attention of the action tokens is also nearly identical between blind

and tracking phases (instruction 0.6–0.8 at a few layers, wrist image 0.2–0.3, agent image 0.1–0.3 in early layers), so the failure is not that the action tokens do not look at the image. $\pi_{0.5}$ knows where the target and the container are; at the moments that matter, its action does not depend on it.

4. Decision-Time Counterfactual Supervision

Section 3 suggests what a remedy has to do: not add information, but create, at the states where the task is decided, pressure to act on the information that is there. Behavior cloning on fixed layouts never creates this pressure, since the memorized action is always correct. We therefore supervise the policy in counterfactual worlds built at its own decision states, where the memorized action is wrong and the correct one is known.

On-policy distillation. The student is $\pi_{0.5}$ with LoRA adapters [12] (rank 32) on the linear layers of the VLM and the action expert; the teacher is the same checkpoint with the adapters disabled. Each iteration, the student drives fresh rollouts, and at every queried state s_t the teacher labels the 50-step chunk A_t (DAGger-style [21]). The student is trained with the flow-matching loss of $\pi_{0.5}$ on these labels,

$$\mathcal{L}_{\text{fact}} = \|v_{\theta}(o_t, x_{\tau}, \tau) - (\epsilon - A_t)\|^2, \quad x_{\tau} = \tau\epsilon + (1 - \tau)A_t. \quad (2)$$

Without counterfactuals this is our matched control (“Distill”); it leaves $\pi_{0.5}$ ’s behavior essentially unchanged.

Counterfactual worlds. At a state s_t , the simulator renders a world s'_t that differs in one respect, chosen by the phase of the episode:

- *Retarget* (before the grasp): the target is moved to a free spot, or swaps places with another object; the hand stays. Label: a privileged scripted approach to above the target’s new position, valid for its first n steps (the grasp is left to the policy).
- *Co-shift* (before the grasp): hand and target move together, so the hand–target geometry is unchanged and the teacher’s nominal chunk remains correct up to the grasp.
- *Relocate* (while carrying): the container, or the furniture region the object goes to, is moved; hand and object stay. Label: a privileged scripted placer to the new location, valid until shortly after the release.

The counterfactual observation o'_t and label A'_t (with a validity mask w over chunk steps) enter the same loss with the same noise and time,

$$\mathcal{L} = \mathcal{L}_{\text{fact}} + \frac{\sum w \odot \|v_{\theta}(o'_t, x'_t, \tau) - (\epsilon - A'_t)\|^2}{\sum w}, \quad (3)$$

on at most 15% of the states of an iteration. A pairwise

consistency term on $v_{\theta}(o') - v_{\theta}(o)$ brought no gain and is not used, in line with [7].

Keeping the labels valid. A scripted teacher with privileged state is only as good as the assumptions behind it, and most of the engineering of the method went into recognizing where they fail. Three automatic checks, all task-agnostic, decide which pairs are kept:

- *Uniqueness.* An object (or furniture) with a twin of the same class in the scene is never moved: the instruction can then only refer to it by where it is (“the bowl next to the plate”), and moving it would change which object is meant.
- *Agreement.* In the factual world, the scripted teacher must head where the $\pi_{0.5}$ teacher heads (cosine at least 0.5 between their intended displacements over 25 steps). This rejects states where the task starts with another step, or where the scripted notion of target or container is wrong.
- *Phase.* Pre-grasp pairs are placed only before the episode’s first grasp, and relocate pairs only while an object of the task is held (gripper closed, object lifted and between the fingers).

Objects are recolored at random during the rollouts (with probability 0.5 per episode) to counter a reliance on color that appears once the policy starts to use the image (Sec. 5.4).

Training budget. 20 iterations of 1024 states each, batch 8, learning rate $5 \cdot 10^{-5}$ with cosine decay, one suite at a time; every compared method uses the same teacher, rollouts, budget and optimizer. Details and the scripted teachers are given in the supplementary material.

5. Experiments

Benchmarks. We train on one LIBERO suite at a time [16] and evaluate on its nominal tasks and on the LIBERO-PRO perturbations of the same suite [26]: *swap* (objects exchange places), *position* (the target is moved; LIBERO-Object only), *object* (objects recolored or re-textured) and *language* (instruction rephrased). Each cell is 10 tasks $\times$ 20 initial states; horizons are 280 (Object), 220 (Spatial), 300 (Goal) and 520 (Long) steps. Seeds are 7, 8, 9 unless stated.

Baselines, all in the same harness, teacher and budget. *Distill*: the on-policy distillation of Sec. 4 without counterfactuals. *ECT* [7]: each iteration, 16 successful episodes are replayed in a transformed scene (robot and start pose unchanged; Object: mirror in y , mirror in x and in xy with a shift; Spatial and Goal: mirror in y ; Long: a shift; furniture is reflected so that it faces the work area); a tracking controller follows the transformed end-effector path, and

successful replays are paired with the original queries, on at most 15% of the states as for DCS. Pairs on 50% of the states do not help ECT (Object swap 25.0, Spatial 48.0). 3D-POINT [22]: the 3D displacement from the gripper to the current sub-goal (the target until it is lifted, then its goal region), taken from the simulator at training *and* test time, is mapped by an MLP with a zero-initialized last layer and added to the flow-time conditioning of every action-expert layer; the teacher does not see it. Published numbers of these methods come from different training regimes (four suites jointly, 30k steps, or 69 tasks) and are not comparable to a per-suite adaptation; we therefore compare re-implementations.

5.1. Main results

Table 2 shows the outcome on the two suites where the swap failure is largest. DCS raises Object swap from 18.7 to 54.5 and Object position from 12.2 to 38.2, and Spatial swap from 40.0 to 65.0, keeping nominal success at 97–99. Under the same teacher and budget, ECT gains 5.1 and 10.8 points on the swap cells, DCS 35.8 and 25.0; the difference to ECT is 30.8 points on Object swap, 21.7 on Object position and 14.3 on Spatial swap. 3D-POINT does not move any cell. The cost is on the *object* cell of LIBERO-Object, where recolored or re-textured objects drop from 92.5 to 81.8; we analyze this trade-off in Sec. 5.4.

Which counterfactual matters. Table 3 varies the counterfactual and keeps everything else fixed. Transforming the whole scene, by mirroring with the equivariantly mapped action or by rotating the world about the robot, changes little, consistent with the per-suite ablations of ECT itself [7]. Co-shifting hand and target, which preserves the geometry the memorized action relies on, does not help either. What helps is a counterfactual that changes the decision: moving the target before the grasp fixes the choice of object (swap 53.0) but not the placement, and moving the container while carrying fixes the placement (position 32.8) but not the choice; combining them fixes both. The label matters as much as the world: on Spatial, labeling the relocated world with $\pi_{0.5}$'s own chunk instead of the privileged placer gives swap 40.0 (= Distill), against 64.7 with the placer (3 seeds), because $\pi_{0.5}$, as Sec. 3 shows, does not act on the relocation it sees.

5.2. What changes inside the policy

Table 4 applies the probes of Sec. 3 to the trained policies. The encoding is nearly the same in the three models. What differs is use: ECT leaves it at the level of $\pi_{0.5}$, while DCS makes the action follow a swap (0.03 $\rightarrow$ 0.53), a renamed target (0.02 $\rightarrow$ 0.54) and a moved container (0.06 $\rightarrow$ 0.44–0.69). Counterfactual supervision thus does not teach the policy to see; it teaches it to act on what it sees. One blind

spot is shared by all three models: early in the approach, the action follows a moved target by only 0.13–0.30.

Explicit geometry is ignored. 3D-POINT is the most direct test of whether information is missing: the policy receives the exact displacement to the sub-goal, including after a swap. Its use is identical to that of $\pi_{0.5}$ (Tab. 4). The injected branch converges to a near-constant offset: its output has a norm of about 1.6 but changes by only 0.05–0.08 when the displacement changes by 10 cm. On fixed layouts the goal point always coincides with the memorized trajectory, so nothing forces the policy to read it. Adding the counterfactual data of DCS to 3D-POINT (with a $10\times$ smaller learning rate for the branch, which otherwise destabilizes training) gives Object swap 43.0 and position 41.5, no better than DCS alone, and the branch is still ignored ($\|f(\mathbf{d}) - f(\mathbf{0})\|$ 0.03–0.12 for 10–30 cm against $\|f(\mathbf{0})\|$ 1.08): under pressure to use the target's position, the policy solves the task through its VLM rather than through the new channel.

5.3. Where the change lives

With the counterfactual data fixed, we restrict which LoRA adapters may change (Tab. 5). The readout of the action expert, the natural place for a decoding bottleneck, gains almost nothing. Forcing the change into the last layers of the action expert gains part of the swap cell but breaks nominal behavior (82.5). The full action expert recovers about 40% of the swap gain and two thirds of the position gain. Adapting only the VLM, with the action expert frozen, recovers 89% of the swap gain at nominal 96, replicated over three seeds (50.0, 44.0, 54.0); most of it comes from layers 9–17 (41.0). Yet the change is not a re-presentation of what the action expert reads: adapting only the keys and values of the VLM, the exact interface the expert attends to, gives nothing and damages every cell. The fix requires the VLM to *process* the scene differently in its mid-to-late layers: deciding which object to go to is computed in the VLM, and $\pi_{0.5}$ learned to compute it from the instruction alone. Placement (position) needs both halves. The appearance trade-off also originates in the VLM: adapting only the VLM yields the lowest *object* cell (68.7), the action expert the highest (87.5).

5.4. Costs, other suites, and an evaluation detail

Appearance. A policy that ignores the scene is robust to recoloring for free; a policy that looks at it can start to identify objects by color. In the VLM-only configuration, the validity checks of Sec. 4 raise the *object* cell from 69 to 83.5; recoloring objects during the rollouts adds only 1.5 points to it and costs 9 points of swap (48.5 $\rightarrow$ 39.5), so its benefit is doubtful. **The main model uses recoloring at probability 0.5; ECT with the same recoloring, and the main**

Method	seeds	LIBERO-Object					LIBERO-Spatial			
		nominal	swap	position	object	language	nominal	swap	object	language
Distill (no counterfactual)	3 / 2	99.3	18.7 $\pm$ 2.8	12.2 $\pm$ 1.3	92.5	99.2	97.5	40.0 $\pm$ 2.8	98.5	96.5
3D-POINT [22] (oracle point)	1	98.5	18.5	12.5	93.0	100.0	99.0	40.0	98.0	97.0
ECT [7]	2+1	99.0	23.8 $\pm$ 0.4	16.5 $\pm$ 0.0	93.3	98.5	98.8	50.8 $\pm$ 1.8	98.0	96.8
DCS (ours)	3	97.3	54.5 $\pm$ 3.8	38.2 $\pm$ 5.6	81.8 [†]	98.0 [†]	98.8	65.0 $\pm$ 1.8	98.8 [†]	96.8 [†]

Table 2. **LIBERO-PRO success (%) after per-suite adaptation of $\pi_{0.5}$** , mean $\pm$ std over seeds (200 episodes per cell and seed). All methods share the teacher, rollouts, budget and optimizer. The untrained $\pi_{0.5}$ scores 99 / 18.5 on Object nominal / swap. Distill has 3 seeds on Object and 2 on Spatial. [†]: 2 seeds, third running.

Counterfactual (LIBERO-Object)	seeds	swap	position	nominal
none (Distill)	3	18.7	12.2	99.3
<i>whole scene transformed</i>				
mirror (equivariant label)	2	20.8	12.8	98.0
rotation about the robot	3	29.2	16.2	99.2
<i>hand-target geometry preserved</i>				
hand and target co-shifted	3	24.3	20.7	98.7
<i>decision changed, privileged label</i>				
retarget (before grasp)	3	53.0	20.5	99.0
relocate (while carrying)	2	22.5	32.8	98.8
retarget + relocate	3	56.3	23.7	96.8
retarget/co-shift + relocate	1	53.0	39.0	96.0

Table 3. **Which counterfactual helps.** Only counterfactuals that change what the policy must do at a decision state, labeled by the privileged teacher, raise the cells: retargeting fixes *which* object (swap), relocating fixes *where* it goes (position). Without validity checks or recoloring.

Probe (LIBERO-Object)	$\pi_{0.5}$	3D-POINT	ECT	DCS
early, target moved	.13-.22	.12-.21	.14-.23	.20-.30
mid, moved 20 cm	.31	.32	.37	.67
mid, swap	.03	.03	.07	.53
mid, target renamed	.02	.02	.01	.54
carrying, cont. $\geq$ 12 cm	.06	.06	.11	.44-.69
use				
target moved $\geq$ 12 cm	.58	—	.62	.65
swap	.55	—	.56	.70
container moved $\geq$ 12 cm	.80	—	.86	.85
enc.				

Table 4. **Use and encoding after training** (seed 7; DCS: the configuration of the last row of Tab. 3). Use: median follow of the action (Sec. 3); ranges span displacement sizes. Encoding: in-state probe R^2 of the action tokens, late expert layers. Every model encodes the change; only DCS acts on it.

Trainable part	params	swap	pos.	nominal	object
none (Distill)	—	18.7	12.2	99.3	92.5
expert readout	34K	23.5	20.5	100	92.5
expert, last 6 layers	4.7M	34.5	30.5	82.5	80.5
action expert (2 seeds)	13.9M	32.0	29.8	94.3	87.5
VLM keys and values	2.7M	20.0	9.5	72.0	63.5
VLM layers 9–17	19.6M	41.0	28.5	92.0	76.5
VLM (3 seeds)	39.2M	49.3	29.8	96.0	68.7
all		53.0	39.0	96.0	80.0

Table 5. **Localization.** LIBERO-Object, the counterfactual data of the last row of Tab. 3, only the listed LoRA adapters trained. Adapting the VLM with the action expert frozen gives 89% of the swap gain.

Method (seed 7)	Goal		Long	
	nominal	swap	nominal	swap
Distill	97.5	29.0	95.0	8.5
ECT	96.5	24.5	84.0	14.0
DCS	89.5–94.0	28.5–31.0	84.0–87.0	11.0–17.0

Table 6. **Multi-step suites.** Ranges over the DCS configurations we tried. **Re-evaluation with fixed furniture placement running.**

LIBERO-Goal and LIBERO-Long. On the multi-step suites (Tab. 6) neither method improves swap much, and both lose nominal success on Long. Per-task analysis traced the Goal drop to the two drawer tasks, and, within them, to two causes. First, $\pi_{0.5}$ opens the top drawer by pushing its handle without closing the gripper, so a gripper-based phase rule took the drawer phase for the approach to the bowl; segmenting phases by when the rest of the scene stops changing removes these pairs, but does not remove the drop. Second, LIBERO *re-draws the furniture placement at every reset*: the cabinet moves by up to 2 cm in x and 1.4 cm in y , and the benchmark’s initial states, which store joint positions only, do not fix it. Pinning the cabinet at controlled offsets shows that DCS, unlike $\pi_{0.5}$ and Distill, is sensitive to it on the top-drawer task (Tab. 7): having learned to act on where movable objects are, it lost part of $\pi_{0.5}$ ’s ability to follow

model without it, are running. With the oracle point of 3D-POINT added, the *object* cell returns to 92.5, since the point identifies the target regardless of its color; a non-privileged grounding source would be needed to claim this.

Cabinet offset	$\pi_{0.5}$	Distill	ECT	DCS
none	90	95	90	80
-1 cm in x	95	95	80	65
-2 cm in x	85	90	85	60
$(-2, -1.4)$ cm	95	90	85	55

Table 7. **Furniture sensitivity.** Goal task “open the top drawer and put the bowl inside” (20 trials, furniture pinned at the placement of a fresh environment plus the offset; offsets within the range LIBERO itself draws).

the furniture. Counterfactuals that nudge the furniture and keep the teacher’s own response there, which $\pi_{0.5}$ handles reliably within this range, are running.

Evaluation protocol. The re-drawn furniture has a second consequence. The standard LIBERO loop runs the trials of a task sequentially in one seeded environment, so trial k sees the furniture of the k -th draw, while evaluation scripts that spread trials over parallel workers give the same trial a different placement depending on scheduling. With one adapter, the drawer task of LIBERO-Goal scores 70% when its 20 trials run sequentially and 95% when they run in freshly built environments, failing on exactly the same trials in repeated sequential runs. All numbers in this paper use the same scheduler for every method; for strict comparability we seed the environment per (task, trial) before each reset, which fixes the furniture of every trial independently of scheduling, and recommend reporting it. LIBERO-Object has no furniture and is unaffected.

6. Conclusion

$\pi_{0.5}$ fails on rearranged scenes not because it cannot see where objects are, but because it does not act on it at the moments that decide the task. Separating use from encoding with decision-time interventions makes this visible, explains why adding geometric information does not help, and points to the remedy: supervision in counterfactual worlds built at those moments, with labels that are valid there. The change it induces lives in the VLM’s mid-to-late layers rather than in the action pathway.

Limitations. Our labels come from privileged scripted teachers in simulation; they cover reaching and placing, not articulated or contact-rich steps, and their validity checks were refined by per-task failure analysis on the benchmark itself. Results are for one policy ($\pi_{0.5}$), one benchmark family, and per-suite adaptation with a small budget; the multi-step suites are not improved, and using the scene costs robustness to appearance changes. The early-approach blind spot, shared by every model we measured, remains open, as does an intervention that targets

the localized layers directly instead of adapting them with data.

References

- [1] Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos Garea, Matthieu Geist, and Olivier Bachem. On-policy distillation of language models: Learning from self-generated mistakes. In *International Conference on Learning Representations (ICLR)*, 2024. 3
- [2] Anonymous. Encoded but not in control. *arXiv preprint arXiv:2610.06235*, 2026. TODO verify full title and authors. 2
- [3] Anonymous. Not all features are created equal. *arXiv preprint arXiv:2603.19233*, 2026. TODO verify full title and authors. 2
- [4] Lucas Beyer, Andreas Steiner, André Susano Pinto, Alexander Kolesnikov, Xiao Wang, et al. PaliGemma: A versatile 3B VLM for transfer. *arXiv preprint arXiv:2407.07726*, 2024. 3
- [5] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024. 1, 2
- [6] Anthony Brohan et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning (CoRL)*, 2023. 1, 2
- [7] Chen et al. When instructions retrieve trajectories: Diagnosing and mitigating generalization failures in VLA models. *arXiv preprint arXiv:2609.39971*, 2026. TODO verify authors. 2, 3, 4, 5, 6, 1
- [8] Dian Chen, Brady Zhou, Vladlen Koltun, and Philipp Krähenbühl. Learning by cheating. In *Conference on Robot Learning (CoRL)*, 2019. 3
- [9] Pim de Haan, Dinesh Jayaraman, and Sergey Levine. Causal confusion in imitation learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019. 2
- [10] Senyu Fei et al. LIBERO-Plus: In-depth robustness analysis of vision-language-action models. *arXiv preprint arXiv:2510.13626*, 2025. TODO verify authors. 2
- [11] Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A. Wichmann. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2:665–673, 2020. 2
- [12] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022. 4
- [13] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. OpenVLA: An open-source vision-language-action model. In *Conference on Robot Learning (CoRL)*, 2024. 1, 2

- [14] Moo Jin Kim, Chelsea Finn, and Percy Liang. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*, 2025. 2
- [15] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023. 3
- [16] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, 2023. 1, 2, 4
- [17] Ajay Mandlekar, Soroush Nasiriany, Bowen Wen, Iretiayo Akinola, Yashraj Narang, Linxi Fan, Yuke Zhu, and Dieter Fox. MimicGen: A data generation system for scalable robot learning using human demonstrations. In *Conference on Robot Learning (CoRL)*, 2023. 3
- [18] Physical Intelligence, Kevin Black, Noah Brown, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025. 1, 2, 3
- [19] Silviu Pitis, Elliot Creager, and Animesh Garg. Counterfactual data augmentation using locally factored dynamics. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. 3
- [20] Delin Qu et al. SpatialVLA: Exploring spatial representations for visual-language-action model. In *Robotics: Science and Systems (RSS)*, 2025. 2
- [21] Stéphane Ross, Geoffrey Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011. 3, 4
- [22] Tsai et al. Direct action-head injection of a grounded 3D point. *arXiv preprint arXiv:2606.27663*, 2026. TODO verify full title and authors. 2, 5, 6, 1
- [23] Chuan Wen, Jierui Lin, Trevor Darrell, Dinesh Jayaraman, and Yang Gao. Fighting copycat agents in behavioral cloning from observation histories. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. 2
- [24] Zhengrong Xue, Shuying Deng, Zhenyang Chen, Yixuan Wang, Zhecheng Yuan, and Huazhe Xu. DemoGen: Synthetic demonstration generation for data-efficient visuomotor policy learning. In *Robotics: Science and Systems (RSS)*, 2025. 3
- [25] Haoyu Zhen, Xiaowen Qiu, Peihao Chen, Jincheng Yang, Xin Yan, Yilun Du, Yining Hong, and Chuang Gan. 3D-VLA: A 3D vision-language-action generative world model. In *International Conference on Machine Learning (ICML)*, 2024. 2
- [26] Xueyang Zhou et al. LIBERO-PRO: Towards robust and fair evaluation of vision-language-action models beyond memorization. *arXiv preprint arXiv:2510.03827*, 2025. TODO verify authors. 1, 2, 4

Knowing Where, Not Going There: Vision-Language-Action Policies Encode Displacements They Do Not Act On

Supplementary Material

585 7. Implementation details

586 **Policy and training.** LeRobot port of the $\pi_{0.5}$ LIBERO
587 checkpoint; LoRA rank 32, $\alpha = 16$, no dropout, on all
588 linear layers of the VLM language model and of the ac-
589 tion expert (vision encoder frozen). Adam, learning rate
590 $5 \cdot 10^{-5}$ with cosine decay to $5 \cdot 10^{-6}$, batch 8, gradient
591 clipping. Each iteration collects rollouts of the student (20
592 episodes per batch, 8 parallel environments) until at least
593 1024 queried states are available, samples 1024 of them,
594 and runs one epoch (128 steps). 20 iterations, i.e. 2560 op-
595 timizer steps. Every 4 iterations the adapter is evaluated on
596 10 trials per task (used only for monitoring). Rendering at
597 256×256 for both cameras.

598 **Scripted teachers.** The *approach* teacher moves the hand
599 to a pre-grasp pose above the target (height from the tar-
600 get’s bounding box) with the controller’s maximal transla-
601 tion and the gripper open; its chunk is valid for the steps
602 it takes to get there. The *placer* lifts the held object to
603 a safe height above the container’s rim, moves above the
604 container’s opening (or the site of a furniture region), de-
605 scends and releases; its chunk is valid until 5 steps after
606 the release. Both act in the same action space as $\pi_{0.5}$ (end-
607 effector deltas, gripper command) and are rolled out in a
608 kinematic model to produce 50-step chunks.

609 **Placing counterfactual objects.** A moved target is
610 placed at 8–30 cm from its position (or on the spot of an-
611 other movable object, which takes the target’s spot, with
612 probability 0.5), inside the area the objects occupy, without
613 overlapping any other object’s bounding circle. A relocated
614 container is moved by 15–40 cm, may leave the object area
615 by up to 15 cm, and carries the objects lying in it. Furniture
616 regions (top of a cabinet, a stove burner, a rack) are moved
617 with their whole fixture, for rendering only.

618 **Validity checks.** Uniqueness uses the class name of every
619 object and fixture. Agreement compares the sum of the first
620 25 translation commands of the scripted teacher in the fac-
621 tual world with that of the $\pi_{0.5}$ teacher (cosine ≥ 0.5 ; pairs
622 where both barely move are kept). An *execution gate* that
623 kept relocate pairs only on tasks whose placer succeeds in
624 closed loop in at least 60% of 20 trials was used in earlier
625 versions; it removed the tasks with the largest gains (e.g.
626 Spatial swap 55.0 with the gate, 65.5 without) and is not
627 used.

Compute. One run (one suite, 20 iterations) takes 5–7
hours on one H200 GPU, of which about half is simulation
and rendering.

8. Re-implemented baselines

ECT. Following [7], a transformed scene reflects the free
objects and the furniture standing on the table (in x about
the objects’ centre, in y about the robot base) and/or shifts
them, while the robot and its start pose are unchanged. Fur-
niture is reflected so that it keeps facing the work area.
The end-effector path of a successful episode is mapped by
the same transform and tracked by a controller that applies
the transformed commands as feed-forward and corrects
half of the tracking error at every step; rotation commands
are mapped as pseudo-vectors and the gripper command
is copied. Untransformed replays succeed in 90–100% of
the episodes with this controller. Only successful replays
are kept, and each is paired with the queries of its source
episode. Transforms per suite follow the paper’s choices;
the sizes of the shifts, not given there, are ours (–8 cm in x ;
(–6, 6) cm with the double mirror).

3D-POINT. Following [22]: $\mathbf{d} = \mathbf{p}_{\text{goal}} - \mathbf{p}_{\text{hand}}$ in the
robot frame, where the goal is the target until it is lifted
by 1 cm, then the region named by the task’s BDDL goal
(on/in); an MLP $3 \rightarrow 1024 \rightarrow 1024$ with a zero-initialized
last layer maps $\mathbf{d}$ to a vector added to the flow-time condi-
tioning of every action-expert layer, at every step. $\mathbf{d}$ comes
from the simulator at training and test time. The branch is
trained with the LoRA learning rate; a $10\times$ larger rate made
Object training collapse.

9. Additional results

Label failures found by per-task analysis. Per-task
breakdowns of every cell located the remaining failures of
the scripted labels, all of which we fixed with task-agnostic
rules: (i) instructions that identify an object by its position
 (“the black bowl next to the plate”), fixed by the uniqueness
check; (ii) tasks that begin with another step, e.g. opening a
drawer before placing a bowl, fixed by the agreement check
and by starting pre-grasp pairs only once the rest of the
scene stops changing ($\pi_{0.5}$ pushes the drawer open without
closing its gripper, so gripper events do not mark the step);
(iii) tasks with two objects to carry, where swapping the
target with the second object labels “go elsewhere” while a
valid object is under the hand, fixed by never swapping with

671 another object the task carries. Results of the configuration
672 with (ii) and (iii) on Goal and Long are running.

673 **LIBERO-Plus.** On the robot-initial-state and layout cat-
674 egories of LIBERO-Plus [10] (60 episodes each, LIBERO-
675 Object), DCS scores 88 / 77 and 83 / 80 (layout / robot,
676 seeds 7 and 8), ECT 83 / 78 and 87 / 77, Distill 85 / 83, 83
677 / 83 and 80 / 83: these categories are not affected.

678 10. Furniture placement in LIBERO

679 LIBERO builds each scene from a BDDL file whose place-
680 ment initializer samples the pose of every object and fixture
681 at reset. The benchmark’s initial states then overwrite the
682 joint positions, which hold the free objects and the robot,
683 but not the poses of fixtures attached to the world (cabi-
684 nets, stoves, racks), which live in the model. Consecutive
685 resets of one environment therefore place the furniture dif-
686 ferently: over 20 resets of LIBERO-Goal, the cabinet moves
687 within 1.9 cm in x and 1.4 cm in y , the stove and the rack
688 within 1.5–1.9 cm. A freshly built environment, seeded as
689 in the official evaluation, always uses the first draw. Simu-
690 lation state, controller state and the robot’s trajectory under
691 identical commands are bit-identical between a reused and
692 a fresh environment; only the images differ. Because the
693 official loop evaluates the trials of a task sequentially in one
694 environment, trial k is tied to the k -th draw; parallel evalua-
695 tion with workers that take over trials breaks this tie. Seed-
696 ing the environment’s random stream with $1000 \cdot \text{task} + \text{trial}$
697 before each reset makes the placement a function of the trial
698 alone; we verified that permuting the order of trials leaves
699 every placement unchanged. LIBERO-Object scenes have
700 no furniture.